

RAI-818: INTELLIGENT **MOBILE ROBOTICS**

Road Maps

- **Visibility Graphs**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

Roadmaps



- Roadmaps
- Basic Idea

The Basic Idea

- If we know that many paths are to be planned in the same environment then it would make sense to construct a data structure once and then reuse the data structure to plan subsequent paths quickly.
- The data structure is called a *map* and *mapping* is the task of generating models of *robot* environments from sensor data.
- Mapping is important when robot does not have *a priori* information about its environment and must rely on sensors to gather the information and *incrementally* construct its map
- Three major *mapping* concepts
 - *Geometric*
 - *Topological*
 - *Occupancy Grids*



Defining futures

Roadmaps



- Roadmaps
- Basic Idea

The Basic Idea

- *Geometric*
 - Uses discrete geometric primitives (points, polygons, circles, splines – 2D Cubes, ellipsoids, planes – 3D)
 - Highly efficient – a large region of space can be represented by a small number of parameters
- *Topological*
 - Abstraction of an environment in terms of discrete places with edges connecting them
 - Graph: nodes represent landmarks (places) and edges represent path between places.
- *Occupancy Grids*
 - Value of each pixel corresponds to the likelihood that its corresponding portion in the workspace or configuration space is occupied
 - Map size is a function of size of environment and size of cell and this approach Imposes a priori geometric grid on world



Defining futures

Roadmaps

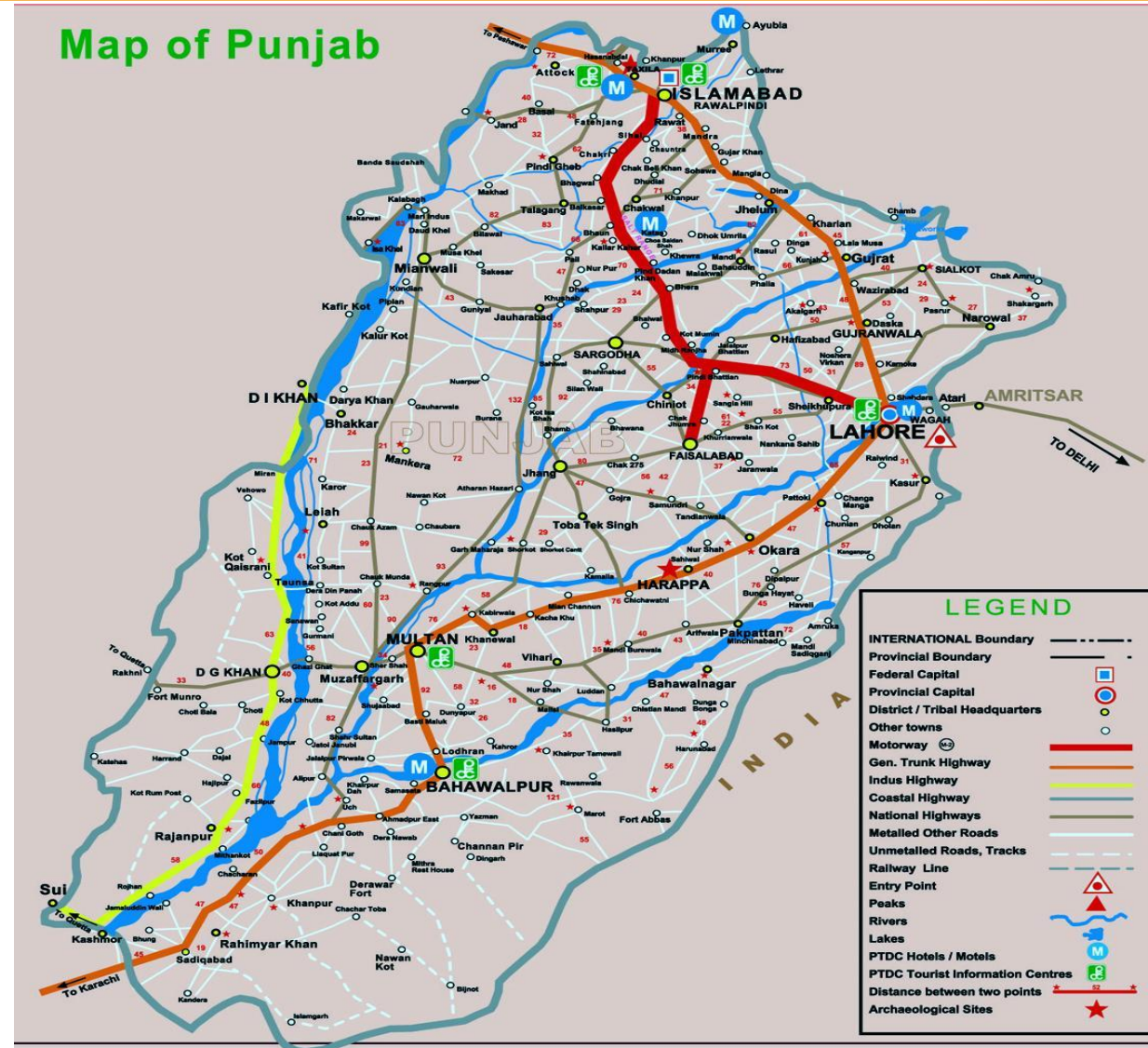


- Roadmaps
- Basic Idea

The Basic Idea

- Roadmaps capture the *connectivity* of Q_{free} by a graph or network of paths.
- Robots use *roadmaps* in the same way as humans use *highways*
- Instead of planning every side street path, we *plan* our path *to the nearest highway*, then we *move along the highway* and finally we *leave the highway* to get to our destination. *Bulk* of the *motion* occurs *on the highway* system

Map of Punjab



Defining futures

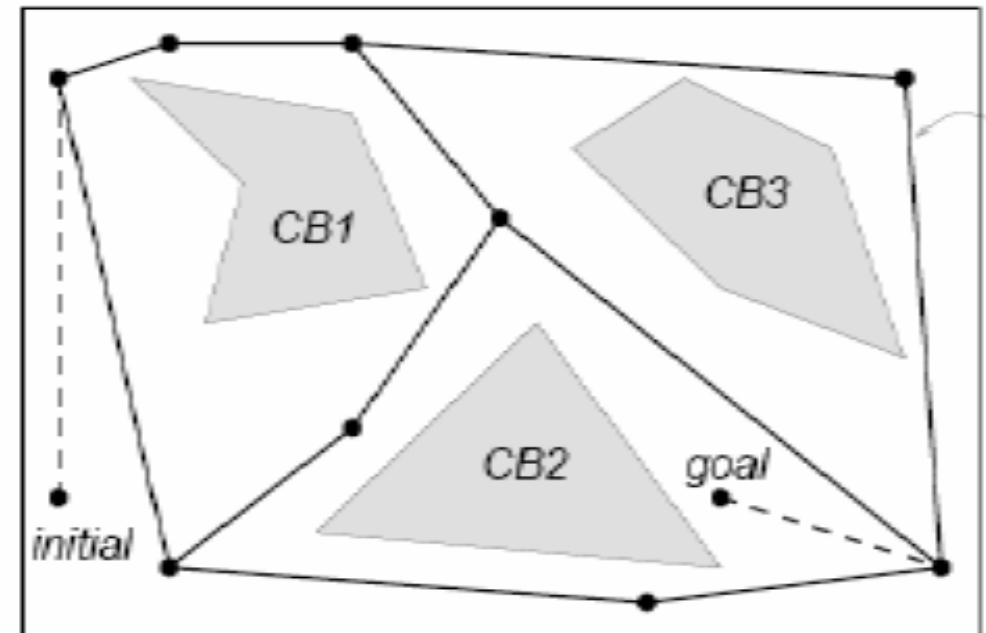
Roadmaps



- Roadmaps
- Basic Idea

The Basic Idea

- Similarly, using a *road map*, the planner can construct *a path between any two points* in a connected component of a robot's free space by first *finding a collision free path* on the roadmap, *traversing the roadmap* to the *vicinity of the goal* and then constructing a *collision free path from* a point on the *roadmap* to the *goal*.
- The *bulk* of the *motion* occurs on the *roadmap*
- *Searching* does *not* occur in a *multidimensional space*
- If the robot *knows* the *roadmap* then it also *knows* the *environment*



Defining futures

Roadmaps



- Roadmaps
 - Basic Idea
 - Definition

Definition

- **Roadmap (RM):** It is a union of curves such that for all start and goal points in Q_{free} that can be connected by a path, the following properties hold:
 - **Accessibility:** There is a path from $q_{start} \in Q_{free}$ to some $q' \in RM$
 - **Departability:** There is a path from some $q'' \in RM$ to $q_{goal} \in Q_{free}$
 - **Connectivity:** There exists a path in RM between q' and q''
 - **One Dimensional**



Defining futures

Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning

Road Map Path Planning

- Build the roadmap
 - nodes are points in Q_{free} (or its boundary)
 - two nodes are connected by an edge if there is a free path between them
- Connect start and goal points to the road map at point q' and q'' , respectively
- Find a path on the roadmap between q' and q'' . The result is a path in Q_{free} from start to goal

Food for Thought: What is the hard part here?



Defining futures

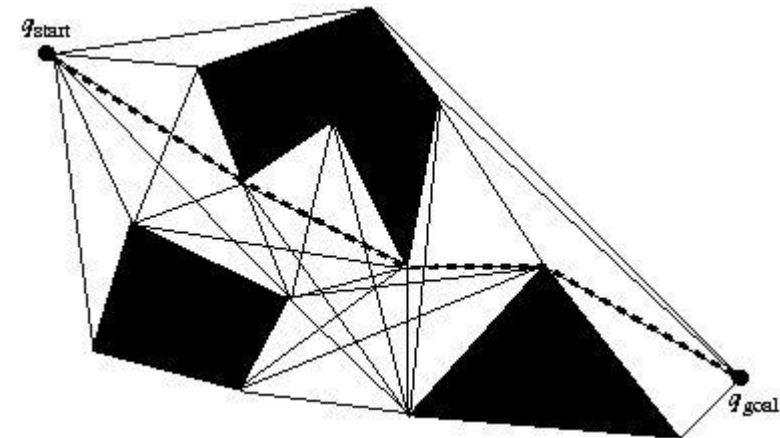
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph Methods

- Defined for polygonal obstacles
- Nodes correspond to vertices of obstacles
- Nodes are connected if
 - they are already connected by an edge on an obstacle
 - the line segment joining them is in free space
- Not only is there a path on this roadmap, but it is the shortest path
- If we include the start and goal nodes, they are automatically connected
- Algorithms for constructing them can be efficient



Defining futures

Roadmaps

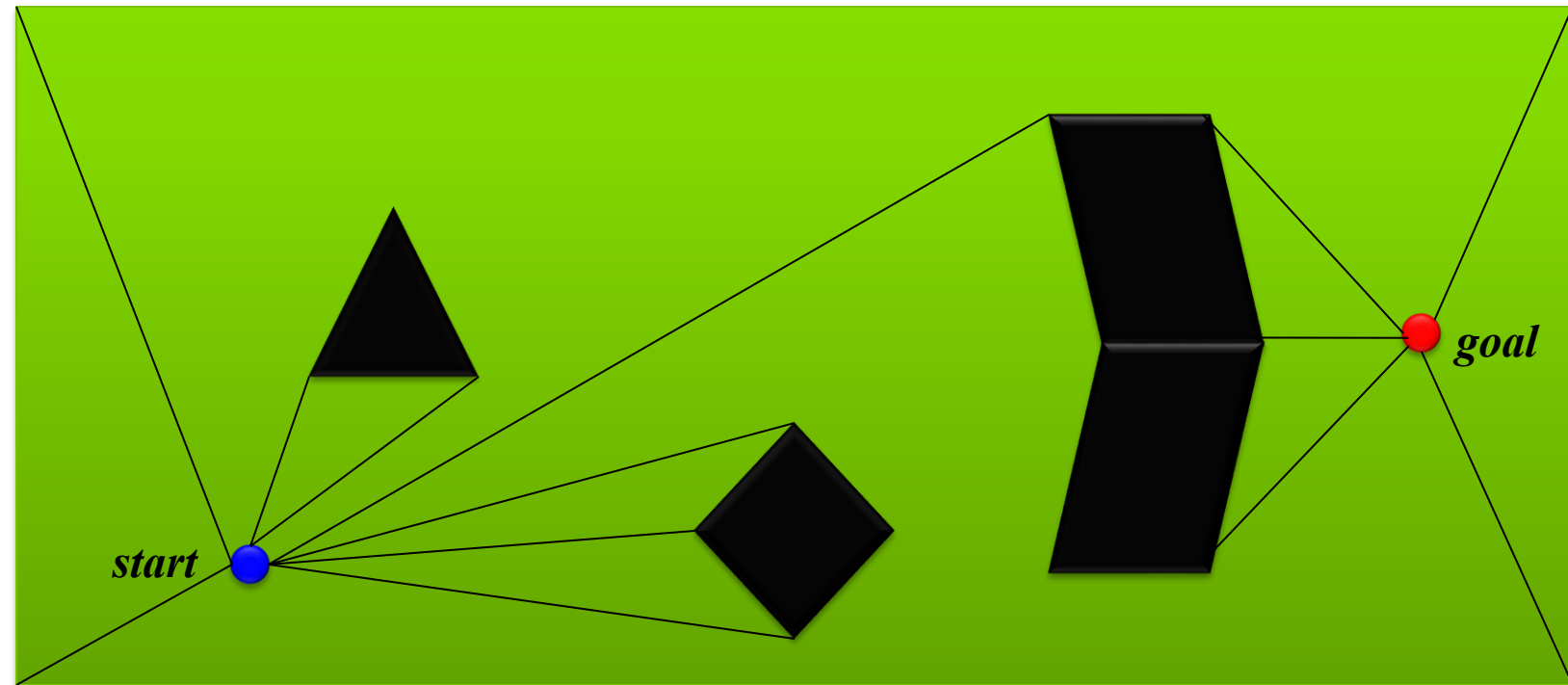


- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part I

- First draw lines of sight from the *start* and *goal* to all *visible* vertices and *corners* of the world

$$e_{ij} \neq 0 \Leftrightarrow sv_i + (1-s)v_j \in Q_{free} \forall s \in [0,1]$$



Defining futures

Roadmaps

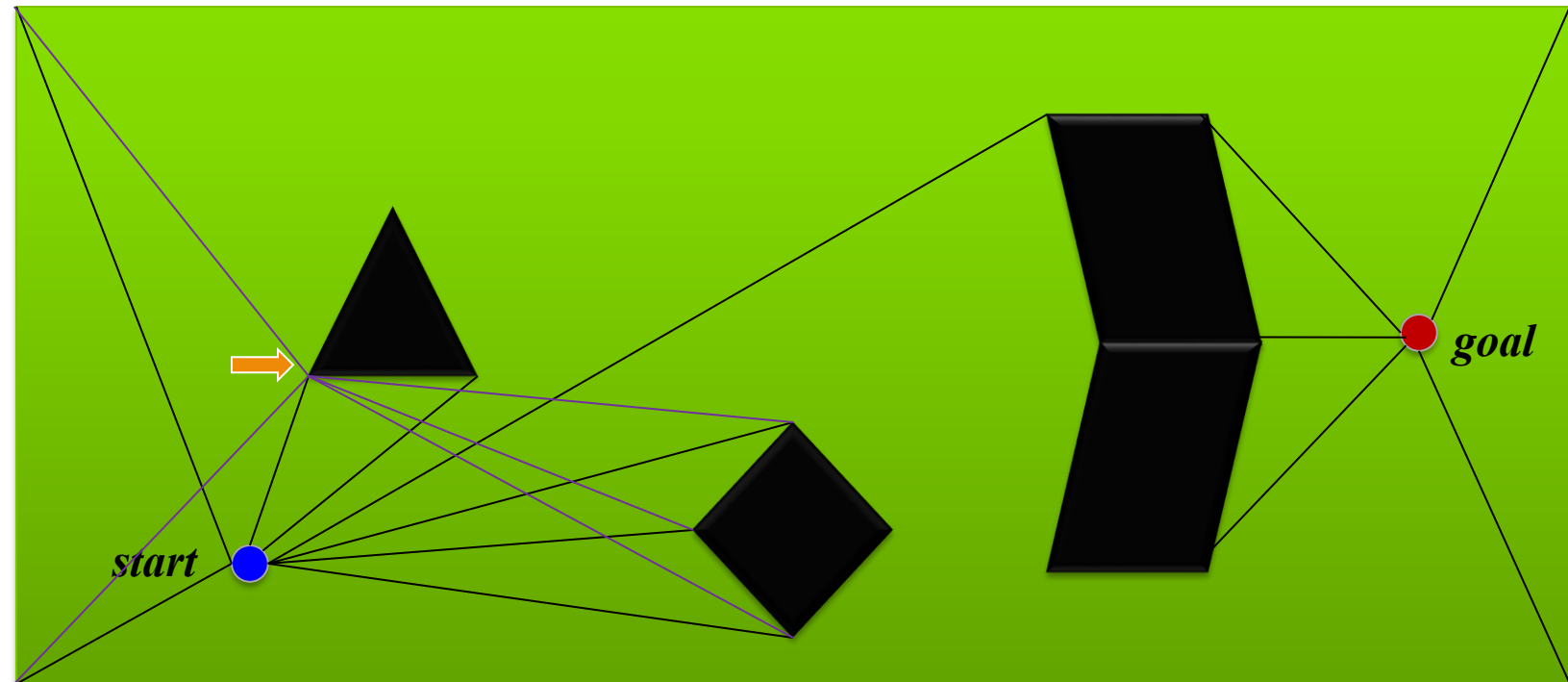


- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part II

- Second, draw lines of sight from every *vertex* of every obstacle like before. Remember lines along edges are also lines of sight

$$e_{ij} \neq 0 \Leftrightarrow sv_i + (1-s)v_j \in Q_{free} \forall s \in [0,1]$$



Defining futures

Roadmaps

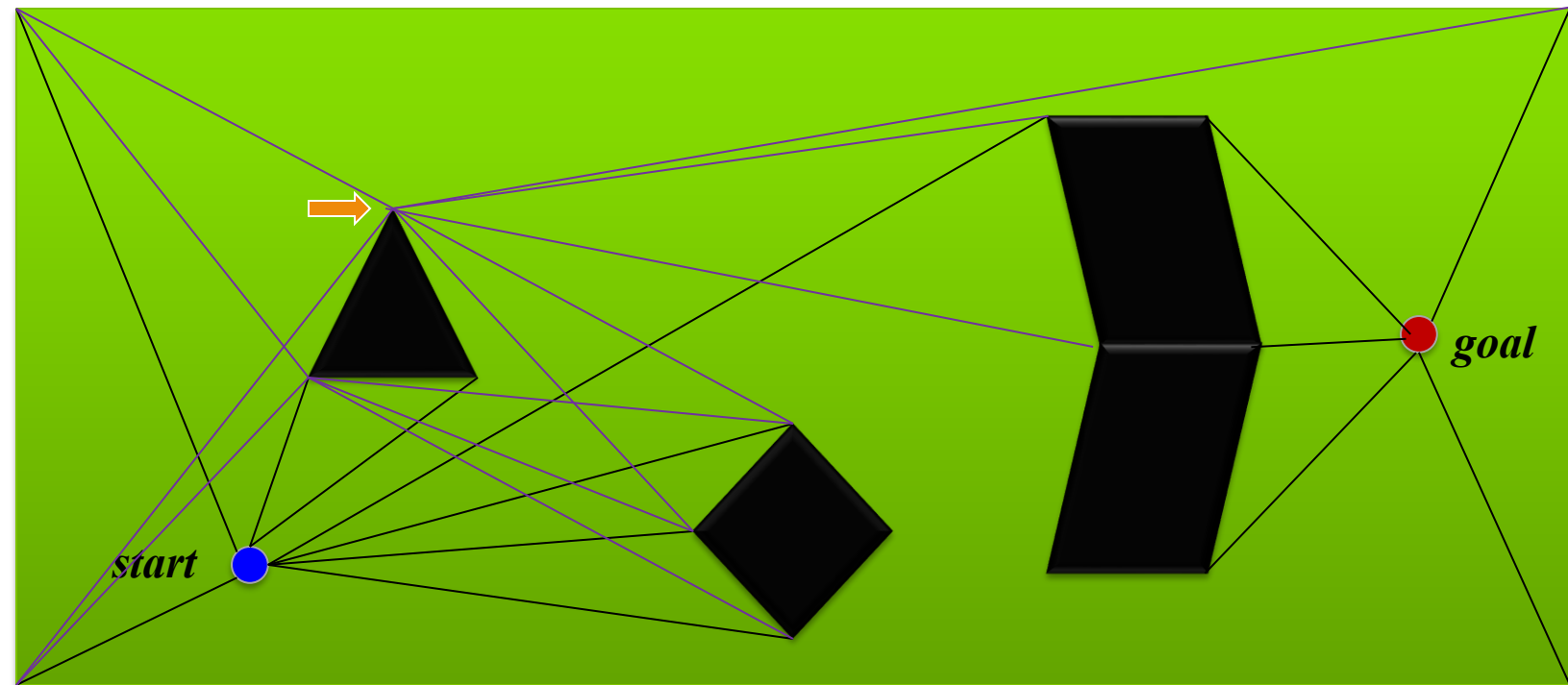


- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part III

- Second, draw lines of sight from every *vertex* of every obstacle like before. Remember lines along edges are also lines of sight

$$e_{ij} \neq 0 \Leftrightarrow sv_i + (1-s)v_j \in Q_{free} \forall s \in [0,1]$$



Defining futures

Roadmaps

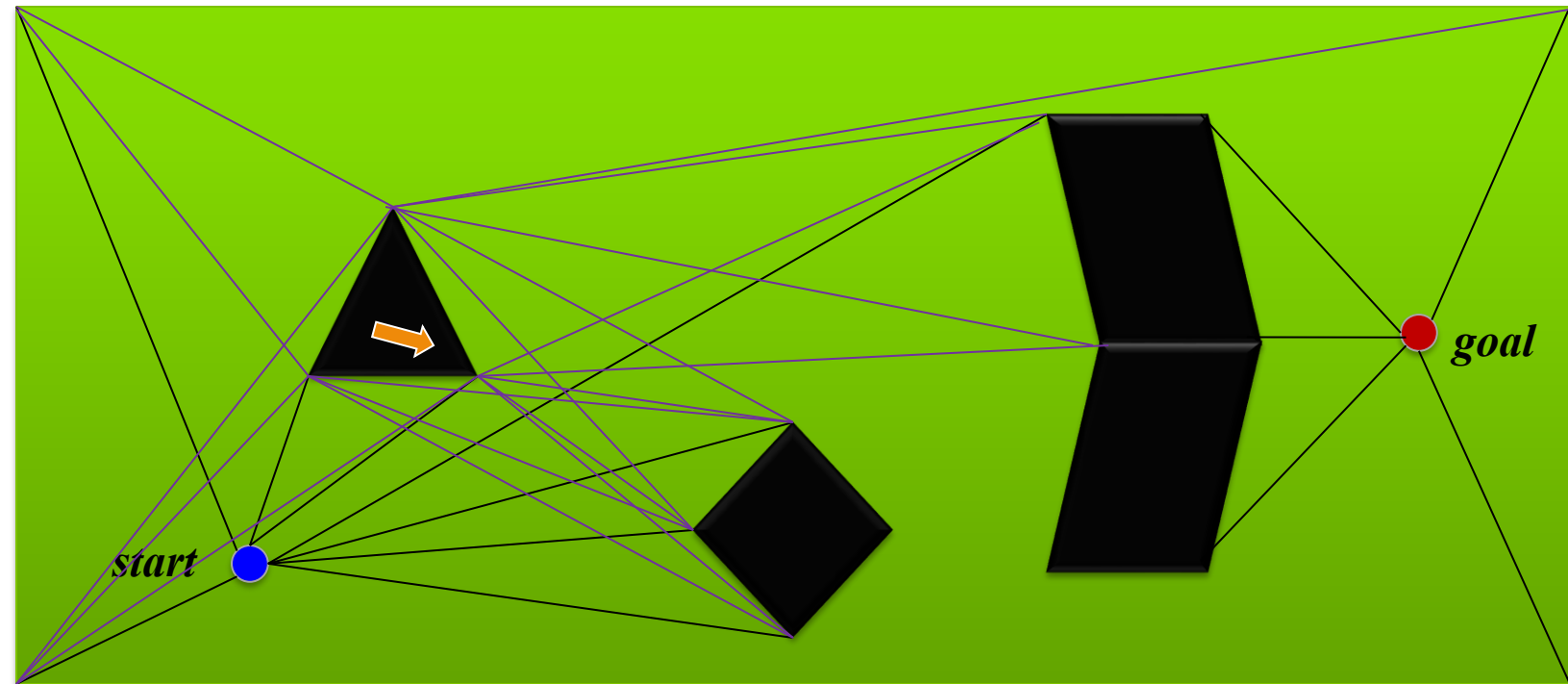


- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part IV

- Second, draw lines of sight from every *vertex* of every obstacle like before. Remember lines along edges are also lines of sight

$$e_{ij} \neq 0 \Leftrightarrow sv_i + (1-s)v_j \in Q_{free} \forall s \in [0,1]$$



Defining futures

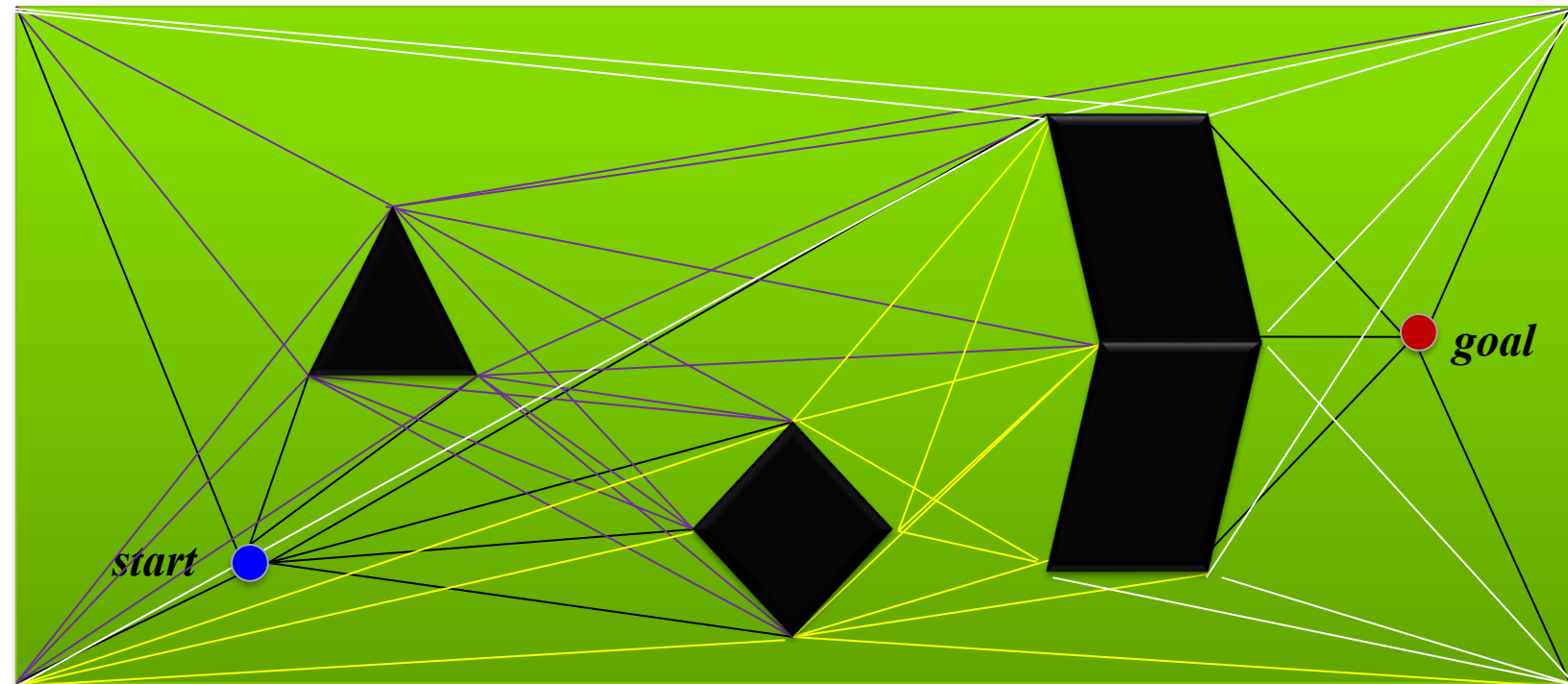
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part IV

- Repeat until done



Defining futures

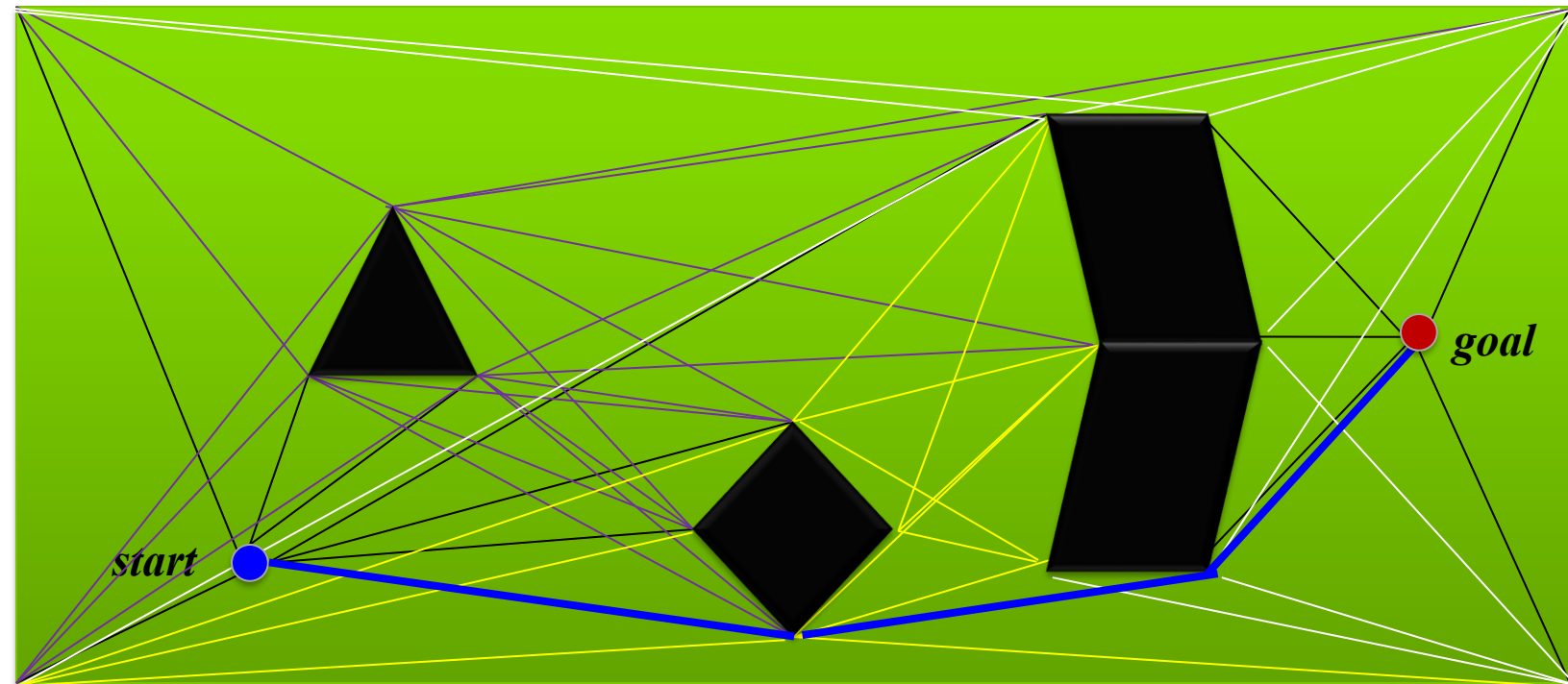
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods

Visibility Graph in Action: Part IV

- Connect start to goal using the shortest path



Defining futures

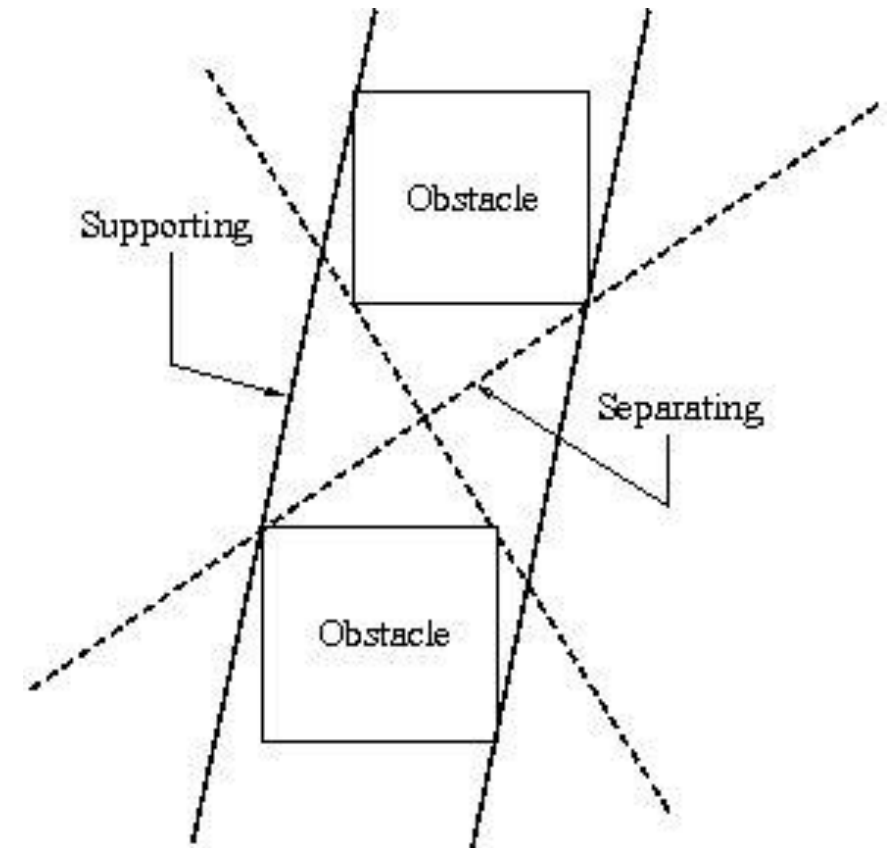
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph

An Improvement Reduced Visibility Graph

- The visibility graph has many needless edges
 - lines to *concave* vertices
 - lines that *head into* the object
- Use of *supporting* and *separating* lines can reduce the number of edges in the graph
- A *supporting* line is tangent to two obstacles such that both obstacles lie on the same side of the line.
- A *separating* line is tangent to two obstacles such that the obstacles lie on the opposite side of the line.



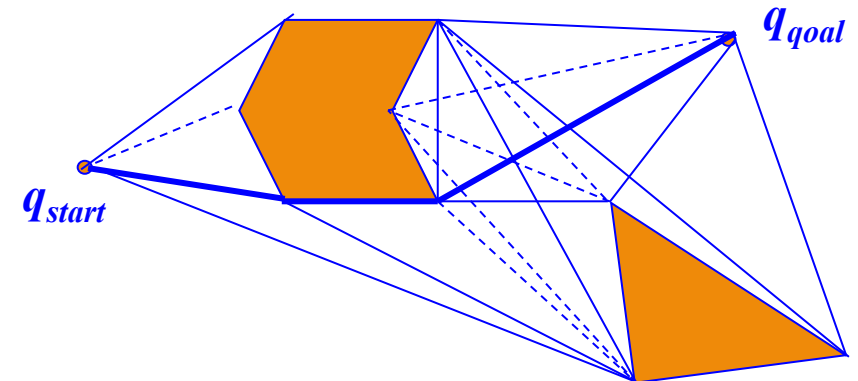
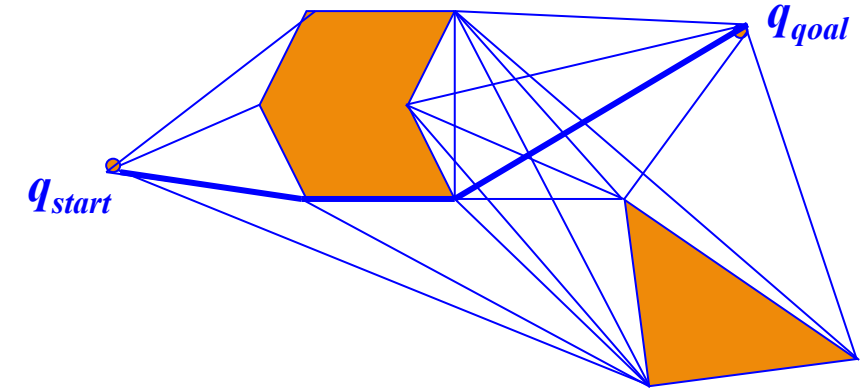
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph

An Improvement Reduced Visibility Graph

- A *reduced visibility graph* is constructed solely from supporting and separating lines
- All edges of a visibility graph that do not lie on a *supporting* or *separating* line are removed
- A *reduced visibility graph* example is as shown
 - No *non-tangent* segment is considered
 - No *concave vertex* is considered



Defining futures

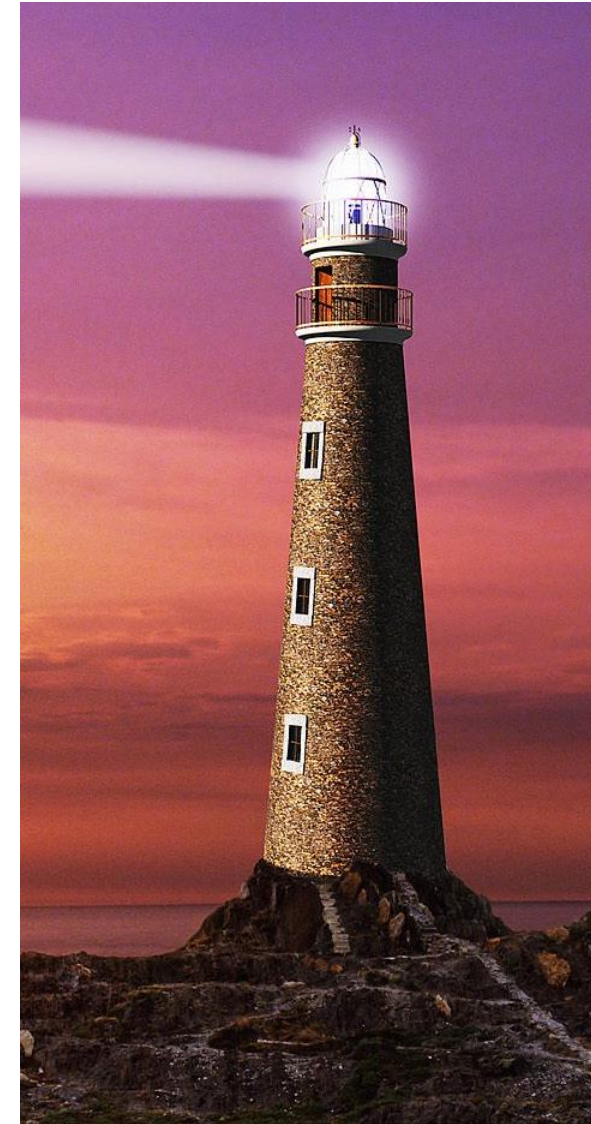
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph
 - + Sweep Line Algorithm

A Further Improvement: Sweep Line Algorithm

- Imagine a rotating beam of light emanating from a light house beacon
- At any point in time the beam illuminates the object that is closest to the lighthouse
- This insight motivates a class of algorithms in computational geometry known as *plane sweep algorithms*
- These algorithms solve a problem by sweeping a *sweep line* across the plane, pausing at each of the vertices of the obstacle
- At each vertex the algorithm updates a partial solution to the problem
- *Plane sweep algorithms* are used to efficiently calculate the intersection of a set of line segments, to compute intersection of polygons



Defining futures

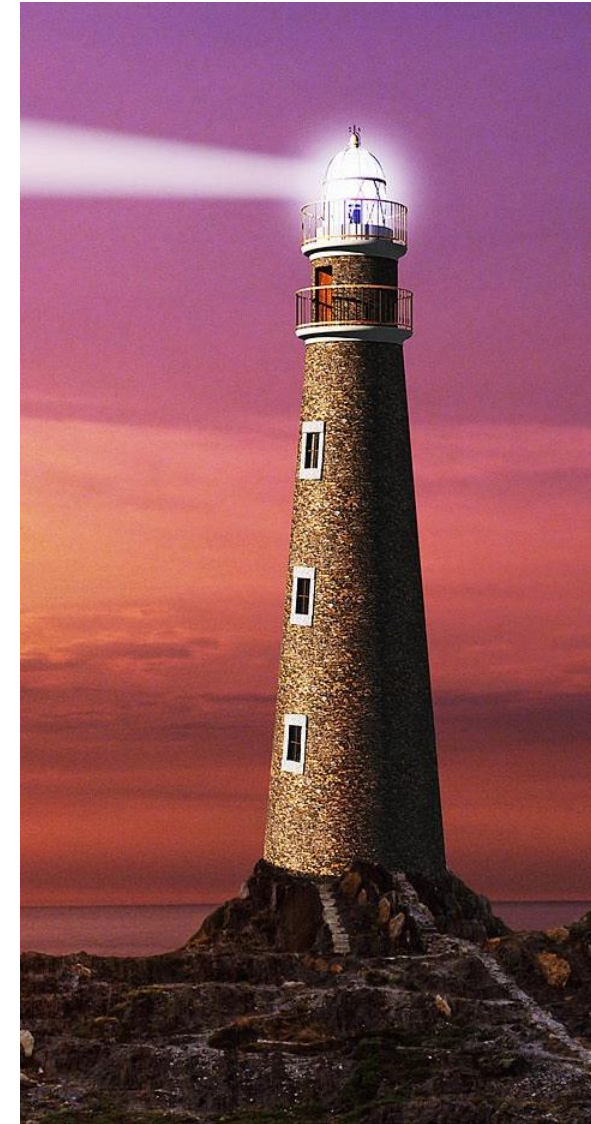
Roadmaps



- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph
 - + Sweep Line Algorithm

A Further Improvement: Sweep Line Algorithm

- For computing the set of vertices visible from v we let the sweep line l be a half line emanating from v and use a rotational sweep rotating l from 0 to 2π
- Key is to incrementally maintain the set of edges that intersect l sorted in order of increasing distance from v
- If a vertex v_i is visible to v then it should be added to the visibility graph
- It is straightforward to determine if v_i is visible to v
 - Let S be the sorted list of edges that intersects the half line emanating from v
 - S is incrementally constructed as the algorithm proceeds
 - If the line segment vv_i does not intersect the closest edge in S and if l does not lie between the two edges incident on v (sweep line does not intersect the interior of the obstacle at v), then v_i is visible to v



Defining futures



Roadmaps

- Roadmaps
 - Basic Idea
 - Definition
 - Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph
 - + Sweep Line Algorithm

Sweep Line More Formally

Input

A set of vertices $\{v_i\}$ (whose edges do not intersect) and a vertex v

Output

A subset of vertices from $\{v_i\}$ that are within the line of sight of v

- 1: For each vertex v_i , calculate α_i , angle b/w horizontal axis and vv_i
- 2: Create the vertex list ε , containing α_i 's sorted in increasing order
- 3: Create the active list S , containing the sorted list of edges that intersect the half-line emanating from v
- 4: **for all α_i do**
- 5: **if v_i is visible v to then**
- 6: Add the edge (v, v_i) to the visibility graph
- 7: **end if**
- 8: **if v_i is the beginning of an edge ε , not in S then**
- 9: Insert the edge ε into S
- 10: **end if**
- 11: **if v_i is the end of an edge ε , not in S then**
- 12: Deleted the edge ε from S
- 13: **end if**
- 14: **end for**



Defining futures

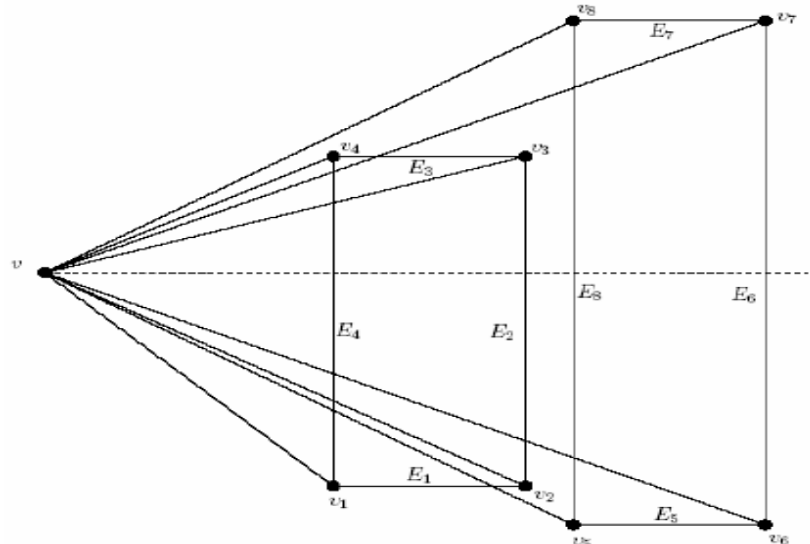


Roadmaps

- Roadmaps
 - Basic Idea
 - Definition
- Path Planning
 - + Visibility Graph Methods
 - + Reduced Visibility Graph
 - + Sweep Line Algorithm

Sweep Line - Example

- The figure shows an example
- Initially calculate the angle α_i 's at which line l will pause (angle of the vertices)
- For each α_i
 - if v_i is visible to v then add vv_i to graph
 - if v_i is the *beginning* of edge ε , insert ε in S
 - if v_i is the *end* of edge ε , delete ε from S



Vert	New S	Actions
start	$\{E_4, E_2, E_8, E_6\}$	Sort edges intersecting l
α_3	$\{E_4, E_3, E_8, E_6\}$	Delete E_2 , Add E_3
α_7	$\{E_4, E_3, E_8, E_7\}$	Delete E_6 , Add E_7
α_4	$\{E_8, E_7\}$	Delete E_3 , and E_4 Add (v, v_4) to graph
α_8	$\{\}$	Delete E_7 , and E_8 Add (v, v_8) to graph
α_1	$\{E_4, E_1\}$	Add E_1 , and E_4 Add (v, v_1) to graph
α_5	$\{E_4, E_1, E_8, E_5\}$	Add E_8 , and E_5
α_2	$\{E_4, E_2, E_8, E_5\}$	Delete E_1 , Add E_2
α_6	$\{E_4, E_2, E_8, E_6\}$	Delete E_5 , Add E_6
end		

